



Опрос про ИИ внутри разработки

[Войти](#)

FilipLinx

6 часов назад

Свой AI-прокси на Java: как я пытался подключиться к Gemini

Средний

12 мин

3.7K

Java*, DevOps*, Искусственный интеллект, Программирование*

Для разработки одного из моих проектов: документа BDoc (Book Document альтернатива устаревшим закрытым бинарным форматами такими как IDML, INDD или Scribus SLA) потребовалась нейросеть, которая может держать контекст очень долго. На данный момент для этой цели идеально подходит Gemini, так как контекстное окно у Gemini в 1 миллион токенов. Но есть одна загвоздка, компания Google заблокировала некоторые регионы. То есть запросы от этих регионов не будут доходить до google. Рассмотрев все за и против, было принято решение написать небольшую программу на java - ai-проху. То есть прокси для нейросетей.

Идея

Написать на java ai-проху, чтобы она работала не только с Gemini, но и если понадобится с OpenAI и Anthropic. У каждого сервиса — свой формат авторизации: у Gemini ключ идёт в query-параметре, у OpenAI — в Authorization: Bearer, у Anthropic — в заголовке x-api-key. Хотелось единой точки входа: один домен, один прокси, а дальше он сам разбирается, куда слать запрос.

Версия 1 — наивная, с захардкоженными ключами

Первая версия была предельно простой: Spring WebFlux + WebClient, роутинг по первому сегменту пути (/gemini/... , /openai/...), и конфиг вида:

```
app:
  providers:
    gemini:
      base-url: https://generativelanguage.googleapis.com
      auth:
        type: query
        parameter: key
        value: ${GEMINI_API_KEY}

    openai:
      base-url: https://api.openai.com
      auth:
        type: bearer
        value: ${OPENAI_API_KEY:}

    anthropic:
      base-url: https://api.anthropic.com
      auth:
        type: header
        header: x-api-key
        value: ${ANTHROPIC_API_KEY:}
```

Объяснить с  SourceCraft

Прокси сам добавлял ключ к запросу — в зависимости от `AuthenticationType(QUERY, HEADER, BEARER)`. Работало. Но у этого подхода два системных недостатка:

Первый. Ключи живут на сервере прокси. Это значит, что прокси становится единой точкой хранения секретов для всех клиентов, которые через него ходят. Если у меня несколько ассистентов с разными правами доступа — разделить их не получится без доработки кода.

Второй, важнее. Любой новый тип авторизации (а он рано или поздно появится — у следующей нейросети может быть кастомная схема) требовал правки Java-кода: новый `enum` в `AuthenticationType`, новая ветка в `switch`, пересборка образа, редеплой.

Версия 2 — pass-through ключей

Следующая идея: прокси вообще не должен знать про ключи. Клиент (тот же OpenCode) уже умеет сам подставлять `Authorization` или `x-api-key` — зачем дублировать эту логику на сервере? Прокси должен просто **прозрачно форвардить все заголовки**, включая авторизационные, как есть.

```

@Service
public class ProxyService {
    private final WebClient webClient;
    private final UrlBuilderService urlBuilderService;

    public ProxyService(WebClient webClient, UrlBuilderService urlBuilderService) {
        this.webClient = webClient;
        this.urlBuilderService = urlBuilderService;
    }

    public Mono<ServerResponse> forward(ProxyRequest request) {
        URI uri = urlBuilderService.build(request);

        WebClient.RequestBodySpec requestBodySpec = webClient.method(request.method())
            .uri(uri)
            .copyHeaders(request.headers(), requestBodySpec);

        WebClient.RequestHeadersSpec<?> clientRequest = hasBody(request.method())
            ? requestBodySpec.body(request.body(), DataBuffer.class)
            : requestBodySpec;

        Mono<ResponseEntity<Flux<DataBuffer>>> responseMono = clientRequest
            .retrieve()
            .onStatus(status -> true, errorResponse -> Mono.empty())
            .toEntityFlux(DataBuffer.class);

        return responseMono.flatMap(entity -> {
            HttpHeaders filteredHeaders = filterHeaders(entity.getHeaders());
            Flux<DataBuffer> body = entity.getBody() != null ? entity.getBody() : null;

            return ServerResponse.status(entity.getStatusCode())
                .headers(h -> h.addAll(filteredHeaders))
                .body(BodyInserters.fromPublisher(body, DataBuffer.class));
        });
    }

    private boolean hasBody(HttpMethod method) {
        return method != HttpMethod.GET && method != HttpMethod.HEAD;
    }

    private HttpHeaders filterHeaders(HttpHeaders source) {
        HttpHeaders filtered = new HttpHeaders();
        source.forEach((name, values) -> {
            if (HttpHeaders.CONTENT_LENGTH.equalsIgnoreCase(name)) return;
            if (HttpHeaders.TRANSFER_ENCODING.equalsIgnoreCase(name)) return;
            if (HttpHeaders.CONTENT_ENCODING.equalsIgnoreCase(name)) return;
        });
        return filtered;
    }
}

```

```

        filtered.put(name, values);
    });
    return filtered;
}

private void copyHeaders(HttpHeaders source, WebClient.RequestBodySpec target) {
    source.forEach((name, values) -> {
        if (HttpHeaders.HOST.equalsIgnoreCase(name)) return;
        if (HttpHeaders.CONTENT_LENGTH.equalsIgnoreCase(name)) return;
        if (HttpHeaders.ACCEPT_ENCODING.equalsIgnoreCase(name)) return;
        values.forEach(value -> target.header(name, value));
    });
}
}

```

Объяснить с 

Это центральный метод, обрабатывающий входящий запрос от клиента и проксирующий его.

Основной метод forward – подготовка запроса к прокси (строки 11–20)

- **Строка 12** – строит итоговый URI с помощью `urlBuilderService.build(request)`.
- **Строка 14** – создаёт спецификацию исходящего запроса (`RequestBodySpec`) с нужным HTTP-методом и URI.
- **Строка 15** – копирует заголовки из входящего запроса в исходящий, исключая служебные.
- **Строки 17–19** – определяет, нужно ли добавлять тело запроса: если метод имеет тело (не GET и не HEAD), то передаёт поток `DataBuffer` из `request.body()`; иначе использует спецификацию без тела.

Выполнение запроса к целевому серверу (строки 21–24)

- **Строка 21** – создаёт реактивный клиентский вызов (`Mono<ResponseEntity<Flux<DataBuffer>>>`).
- **Строка 22** – отправляет запрос и начинает разбор ответа (`.retrieve()`).
- **Строка 23** – указывает, что **на любой HTTP-статус** (включая ошибки) не надо выбрасывать исключение, а нужно передать ответ дальше как есть.
- **Строка 24** – получает ответ как `ResponseEntity` с телом в виде `Flux<DataBuffer>` (поток байтов).

Обработка ответа и формирование `ServerResponse` (строки 26–33)

- **Строка 27** – фильтрует заголовки полученного ответа (удаляет `Content-Length`, `Transfer-Encoding`, `Content-Encoding`).
- **Строка 28** – извлекает тело ответа, если оно есть, или создаёт пустой поток.
- **Строки 30–32** – строит финальный `ServerResponse` :
 - статус – как у ответа от провайдера,
 - заголовки – отфильтрованные,
 - тело – потоково передаётся через `BodyInserters.fromPublisher`.
- Весь блок возвращает `Mono<ServerResponse>`.

Вспомогательный метод `hasBody` (строки 36–38)

- Определяет, может ли HTTP-метод содержать тело. Возвращает `true` для всех методов, кроме `GET` и `HEAD`.

Конфиг провайдера стал предельно простым:

```
public class ProviderProperties {  
    private String baseUrl;  
    // только baseUrl. Больше ничего  
}
```

Объяснить с 

Ключей в конфиге нет вообще. Провайдер описывается одной строкой — куда слать запрос.

Версия 3 — универсальный конфиг без пересборки образа

Мне не хотелось каждый раз лезть в `application.yml` чтобы добавлять конфиг новой нейросети. Нужен был универсальный механизм добавления настроек.

Решение — переносить список провайдеров в JSON-строку внутри одной переменной окружения `PROVIDERS_JSON`, и парсить её при старте приложения:

@Component

```
public class ProviderConfigLoader implements ApplicationRunner {  
    private final AppProperties appProperties;  
    private final ObjectMapper objectMapper;  
  
    public ProviderConfigLoader(AppProperties appProperties, ObjectMapper objectMapper)  
    {  
        this.appProperties = appProperties;  
        this.objectMapper = objectMapper;  
    }  
}
```

@Override

```
public void run(ApplicationArguments args) throws Exception {  
    String json = appProperties.getProvidersJson();  
    if (json == null || json.isBlank()) {  
        return;  
    }  
    Map<String, Map<String, String>> raw =  
        objectMapper.readValue(  
            json,  
            objectMapper.getTypeFactory()  
                .constructMapType(  
                    LinkedHashMap.class,  
                    String.class,  
                    Map.class  
                )  
        );  
  
    Map<String, ProviderProperties> providers = new LinkedHashMap<>();  
    raw.forEach((name, fields) -> {  
        ProviderProperties provider = new ProviderProperties();  
        provider.setBaseUrl(fields.get("baseUrl"));  
        providers.put(name, provider);  
    });  
    appProperties.setProviders(providers);  
}
```

Объяснить с 

Это Spring-компонент, который при старте приложения загружает конфигурацию для различных AI-провайдеров из JSON-строки (хранящейся в AppProperties), парсит её и сохраняет в структурированном виде для дальнейшего использования.

Строка 14: Получает строку JSON из `appProperties.getProvidersJson()`. Эта строка обычно задаётся в `application.yml`

Строка 18: Парсинг JSON-строки в карту `raw` типа `Map<String, Map<String, String>>`.

Строка 19: `objectMapper.readValue()` – десериализация.

Строки 21–26: через `TypeFactory` конструируется сложный тип: внешний `Map` с ключом-строкой (имя провайдера) и значением – внутренним `Map` (строковые поля, например, `baseUrl`). Явно указан `LinkedHashMap`, чтобы сохранить порядок вставки.

Строка 29: Создаётся новая пустая `LinkedHashMap` с именем `providers`. Она будет хранить имя провайдера и объект `ProviderProperties` (куда будут помещены настройки)

Строка 35: Полученная карта провайдеров сохраняется обратно в `appProperties` через метод `setProviders()`. Теперь другие компоненты могут получить доступ к настройкам провайдеров через тот же бин `AppProperties`.

И теперь весь конфиг провайдеров – это одна переменная окружения:

```
{
  "gemini": { "baseUrl": "https://generativelanguage.googleapis.com" },
  "openai": { "baseUrl": "https://api.openai.com" },
  "anthropic": { "baseUrl": "https://api.anthropic.com" }
}
```

Объяснить с 

Появилась новая нейросеть — открыл панель хостинга, дописал в JSON ещё одну пару "имя → baseUrl", перезапустил процесс. Ни строчки Java-кода, ни git commit, ни пересборки образа.

Что получилось в итоге

Роутинг по пути остался предельно тонким — весь смысл прокси:

```
public record ProviderPath(String provider, String path) {

    public static ProviderPath parse(String rawPath) {
        if (rawPath == null || rawPath.isBlank()) {
            throw new IllegalArgumentException("Empty request path");
        }
    }
}
```

```

String path = rawPath.trim();

if (!path.startsWith("/")) {
    throw new IllegalArgumentException("Invalid path: " + path);
}

String withoutFirstSlash = path.substring(1);

int separator = withoutFirstSlash.indexOf('/');

String provider;
String providerPath;

if (separator == -1) {
    provider = withoutFirstSlash;
    providerPath = "/";
} else {
    provider = withoutFirstSlash.substring(0, separator);
    providerPath = withoutFirstSlash.substring(separator);
}

if (provider.isBlank()) {
    throw new IllegalArgumentException("Provider is empty");
}

return new ProviderPath(provider, providerPath);
}
}

```

Объяснить с 

Блок 1 – Валидация входных данных (строки 4–12)

- **Строки 4–6** – проверка, что rawPath не null, не пустой и не состоит из пробелов. Если что-то не так – исключение.
- **Строка 8** – удаление лишних пробелов.
- **Строки 10–12** – проверка, что путь начинается с символа /. Если нет – исключение.

Блок 2 – Подготовка к разбору (строки 14–16)

- **Строка 14** – удаляет первый слеш из строки, чтобы получить часть после него (например, "openai/v1/chat").
- **Строка 16** – ищет позицию первого слеша в этой части, чтобы разделить её на две части.

Блок 3 – Разделение на провайдера и путь (строки 18–27)

- **Строки 18–19** – объявление переменных provider и providerPath.
- **Строки 21–23** – если слеша не было (separator == -1), то вся строка после первого слеша становится именем провайдера, а путь равен "/".
- **Строки 24–27** – если слеш есть, то часть до него – это провайдер, а часть от него (включая сам слеш) – путь провайдера.

Блок 4 – Финальная проверка и возврат результата (строки 29–33)

- **Строки 29–31** – проверяют, что имя провайдера не оказалось пустым (например, при пути "/"). Если пусто – исключение.
- **Строка 33** – создаёт и возвращает объект ProviderPath с вычисленными значениями.

Как это работает:

Входной путь (rawPath)	Результат (provider, providerPath)	Пояснение
"/openai/v1/chat"	("openai", "/v1/chat")	Провайдер – openai, путь – /v1/chat
"/anthropic"	("anthropic", "/")	Провайдер – anthropic, путь по умолчанию – корень
"/"	Исключение: «Provider is empty»	Нет имени провайдера
"openai/v1"	Исключение: «Invalid path»	Путь должен начинаться с /
null	Исключение: «Empty request path»	

Безопасность

Чтобы как-то обезопасить прокси и им мог пользоваться только конкретный пользователь. Нужно было добавить какую-то фильтрацию. Городить UI или API для управления не хотелось. У меня статический IP и идея просто добавить в конфиг выглядела очевидной. Но оставалась проблема: а как я смогу пользоваться прокси, если нужно будет с рабочего места вызвать нейросеть. Или IP будет динамическим. Я нашел решение.

В настройки добавляем ADMIN_TOKEN и мини-эндпоинт `/admin/allow-ip`, который регистрирует текущий IP на несколько часов.

```
app:
  security:
    enabled: true
    allowed-ips: ${ALLOWED_IPS:}
    admin-token: ${ADMIN_TOKEN:}
    dynamic-ip-ttl-minutes: 720
```

Объяснить с 

Но хотелось это сделать изящно, без добавления контроллера вызова. Я обошелся фильтрами.

```
@Configuration
public class RouterConfig {

    @Bean
    RouterFunction<ServerResponse> routes(ProxyHandler proxyHandler, AdminHandler adminHandler) {
        return route(POST("/admin/allow-ip"), adminHandler::allowCurrentIp).and()
    }
}
```

Объяснить с 

`route(POST("/admin/allow-ip"), adminHandler::allowCurrentIp)` – создаёт маршрут для **POST-запросов** по пути `/admin/allow-ip`. При обращении по этому адресу будет вызван метод `allowCurrentIp` у `adminHandler`. Этот эндпоинт, позволяет добавить текущий IP-адрес в белый список.

```
public Mono<ServerResponse> allowCurrentIp(ServerRequest request) {
    String configuredToken = appProperties.getSecurity().getAdminToken();
    if (configuredToken == null || configuredToken.isBlank()) {
        return ServerResponse.status(HttpStatus.SERVICE_UNAVAILABLE).bodyVa
```

```

    }

    String providedToken = request.headers().firstHeader(ADMIN_TOKEN_HEADER);
    if (!configuredToken.equals(providedToken)) {
        return ServerResponse.status(HttpStatus.FORBIDDEN).bodyValue("{\"error\": \"Invalid token\"}");
    }

    InetSocketAddress remoteAddress = request.remoteAddress().orElse(null);
    if (remoteAddress == null || remoteAddress.getAddress() == null) {
        return ServerResponse.status(HttpStatus.BAD_REQUEST).bodyValue("{\"error\": \"Invalid IP address\"}");
    }

    String ip = remoteAddress.getAddress().getHostAddress();
    Instant expiresAt = ipAllowlistService.allow(ip);
    return ServerResponse.ok().bodyValue("{\"allowedIp\": \"" + ip + "\", \"expiresAt\": \"" + expiresAt.toISO8601String() + "\"}");
}

```

Объяснить с 

Внутри метода мы проверяем что токен авторизации задан в настройках, а потом сверяем с тем что пришел. Если не задан или не пришел, то выводим сообщения об ошибке.

Для добавления ip в белый список на 12 часов нужно вызвать:

```

curl --request POST \
  --url https://you_project.onrender.com/admin/allow-ip \
  --header 'X-Admin-Token: you_token'

```

Объяснить с 

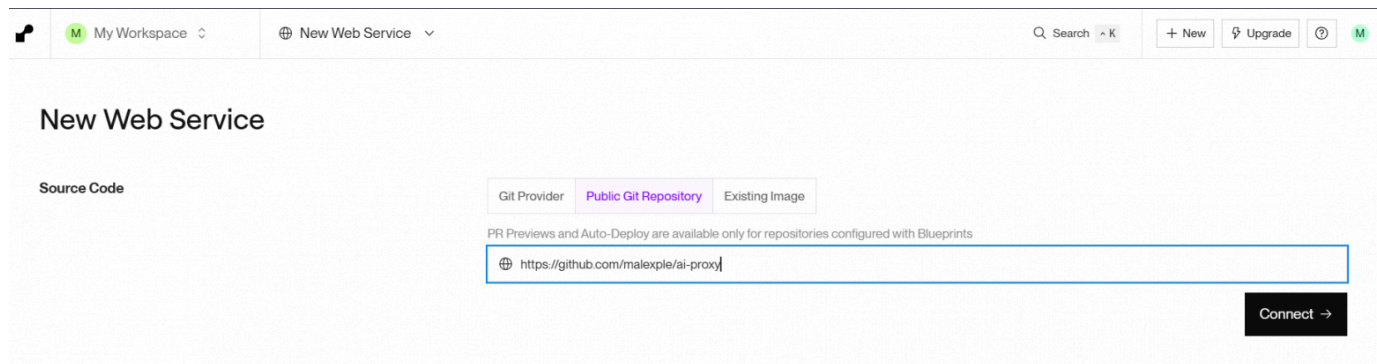
На сколько это удобно покажет время. Но это гораздо проще чем прописывать в настройках сервиса каждый раз IP. И рестартовать сервис.

Настраиваем ai-proxy в render

Мне не хотелось арендовать отдельный VDS или виртуальную машину. И я на мой взгляд нашел довольно интересное решение. Что такое render (www.render.com) это облачная платформа (PaaS), созданная как современная альтернатива Heroku. Heroku в какой то момент выкупила Salesforce и платформа стала полностью платной. Render пока позволяет делать это бесплатно, по крайней мере для нашей задачи хватит ресурсов, которые сервис предоставляет бесплатно.

Тут достаточно все просто. Регистрируетесь в www.render.com. Зайти можно с существующей учеткой от github.

Далее нажимаете New -> Web service. Указывается адрес открытого проекта и нажимаете Connect.



The screenshot shows the 'New Web Service' interface on the Render platform. At the top, there is a navigation bar with 'My Workspace' and 'New Web Service' tabs, along with search and action buttons. The main section is titled 'New Web Service' and features a 'Source Code' label. Below this, there are three tabs: 'Git Provider', 'Public Git Repository' (which is selected), and 'Existing Image'. A text input field contains the URL 'https://github.com/malexple/ai-proxy/'. Below the input field, a small note states: 'PR Previews and Auto-Deploy are available only for repositories configured with Blueprints'. A 'Connect' button with a right-pointing arrow is located at the bottom right of the form.

Далее выбирается free подписку. Можете поменять регион какой вам нужен, и нажимаете Deploy Web Service. Можно даже выбрать требуемый вам регион.

M

My Workspace

New Web Service

Q Search

+ New

Upgrade

M

New Web Service

It looks like you're using **Docker**, so we've autofilled some fields accordingly.

Source Code

malexple / ai-proxy · 2h ago

Edit

Name

A unique name for your web service.

ai-proxy

Project Optional

Add this web service to a [project](#) once it's created.

Create a new project to add this to?

You don't have any projects in this workspace. [Projects](#) allow you to group resources into environments so you can better manage related resources.

+ Create a project

Language

Choose the [runtime environment](#) for this service.

Docker

Branch

The Git branch to build and deploy.

master

Region

Your services in the same [region](#) can communicate over a [private network](#).

Frankfurt (EU Central)

Root Directory Optional

If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a [monorepo](#).

e.g. src

Instance Type

For hobby projects

Free

\$0 / month

512 MB (RAM)

0.1 CPU

For professional use

For more power and to get the most out of Render, we recommend using one of our paid instance types. All paid instances support:

- Zero Downtime
- SSH Access
- Scaling
- One-off jobs
- Support for persistent disks

Starter

\$7 / month

512 MB (RAM)

0.5 CPU

Pro

\$85 / month

4 GB (RAM)

2 CPU

Pro Max

\$225 / month

16 GB (RAM)

4 CPU

Standard

\$25 / month

2 GB (RAM)

1 CPU

Pro Plus

\$175 / month

8 GB (RAM)

4 CPU

Pro Ultra

\$450 / month

32 GB (RAM)

8 CPU

Upgrade to enable more features

Free instances spin down after periods of inactivity. They do not support SSH access, scaling, one-off jobs, or persistent disks. Select any paid instance type to enable these features.

Need a [custom instance type](#)? We support up to 512 GB RAM and 64 CPUs.

Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more](#).

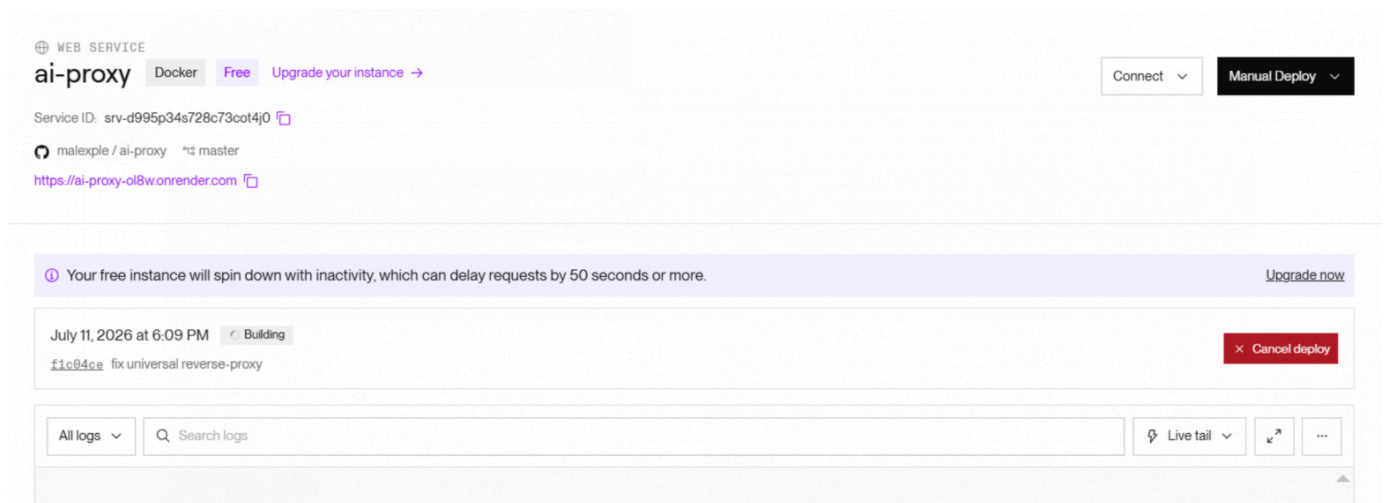
+ Add Environment Variable

Add from .env

> Advanced

Deploy Web Service

Дальше проект соберется и запустится



В проекте уже есть `.dockerfile` и `rende.yaml`. Поэтому переменные среды должны подтянутся сами.

```
services:
- type: web
  name: ai-proxy
  runtime: docker
  plan: free
  autoDeploy: true
  healthCheckPath: /actuator/health
  envVars:
    - key: PORT
      value: "8080"
    - key: ALLOWED_IPS
      sync: false
    - key: ADMIN_TOKEN
      sync: false
    - key: PROVIDERS_JSON
      sync: false
```

Объяснить с 

Если этого не произошло. Зайдите в Environment и добавьте ручную

Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more](#).

KEY	VALUE		
ALLOWED_IPS			
ADMIN_TOKEN			
PROVIDERS_JSON			

[+ Add variable](#)[Save, rebuild, and deploy](#)[Cancel](#)

Настройки в параметр PROVIDERS_JSON требуется вносить без переносов строки.
Примерно так:

```
{"gemini":{"baseUrl":"https://generativelanguage.googleapis.com"},"openai":{"ba
```

[Объяснить с](#)

Далее нажимаете **Save, rebuild and deploy**. И ждете пока сервис запустится с новыми параметрами.

Запрос в Gemini теперь будет такой:

```
curl --request GET \  
  --url https://ai-proxy-you_id.onrender.com/gemini/v1beta/models \  
  --header 'User-Agent: insomnia/8.2.0' \  
  --header 'x-goog-api-key: YOU_GEMINI_KEY'
```

[Объяснить с](#)

Настройка OpenCode для работы с Gemini

Теперь можно в настройках openCode поменять только пути к нейросетям. Остальные параметры остаются те же.

```
{  
  "$schema": "https://opencode.ai/config.json",  
  "provider": {  
    "google": {
```



```

"npm": "@ai-sdk/google",
"name": "Gemini via ai-proxy",
"options": {
  "baseUrl": "https://ai-proxy-ol8w.onrender.com/gemini/v1beta/",
  "apiKey": "YOU_GEMINI_API_KEY"
},
"models": {
  "gemini-2.5-flash": { "name": "Gemini 2.5 Flash" },
  "gemini-2.5-pro": { "name": "Gemini 2.5 Pro" }
}
},
"model": "google/gemini-2.5-flash"
}

```

Объяснить с 

Запуск Docker контейнера

Запустить можно так:

```
docker run -d -p 8080:8080 -e "PORT=8080" --name ai-proxy malexple/ai-proxy:latest
```

Объяснить с 

Но так как JSON с настройками к нейросетям может быть большой, лучше использовать bash скрипт. Чтобы не было проблем с кавычками

```
export PROVIDERS_JSON='{ "gemini": { "baseUrl": "https://generativelanguage.googleapis.com/v1beta1/models/gemini-2.5-flash:generateContent?key=YOUR_API_KEY" } }'
docker run -d -p 8080:8090 -e "PORT=8080" -e PROVIDERS_JSON --name ai-proxy malexple/ai-proxy:latest
```

Объяснить с 

Либо через docker-compose.yml

```

version: "3.8"

services:
  ai-proxy:
    image: malexple/ai-proxy:latest
    container_name: ai-proxy
    ports:
      - "8080:8080"

```



```
env_file:
  - .env
environment:
  PORT: "8080"
  PROVIDERS_JSON: >-
    {"gemini":{"baseUrl":"https://generativelanguage.googleapis.com"},
    "openai":{"baseUrl":"https://api.openai.com"},
    "anthropic":{"baseUrl":"https://api.anthropic.com"}}
restart: unless-stopped
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8080/actuator/health"]
  interval: 30s
  timeout: 5s
  retries: 3
  start_period: 10s
```

Объяснить с 

Что я получил

- Единая точка входа для трёх нейросетей
- Никаких секретов на сервере
- Добавление нового провайдера — без пересборки образа
- Безопасность без UI и без БД
- True streaming без буферизации в памяти.
- Гибкий деплой
- Интеграция с рабочими инструментами (opencode)

Проект лежит тут

Теги: java, spring boot, spring webflux, reverse proxy, gemini api, openai api, anthropic api, docker, webclient, обход блокировок

Хабы: Java, DevOps, Искусственный интеллект, Программирование

Присылаем лучшие статьи раз в месяц

Электронная почта

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



13

Карма

@FilipLinx

Пользователь

Подписаться



 Комментарии 1

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



ndokutovich

22 часа назад

Теория относительности — это синус. Мнимого угла



Сложный



12 мин



16K



+79



81



25



monobogdan

15 часов назад

Полный разбор схемотехники кнопочного телефона. Samsung C100 — маленькое чудо корейской инженерной мысли



Простой



11 мин



11K

Ретроспектива



+31



12



26



Erwinmal

20 часов назад

Онигири: путь рисового колобка от японской древности до наших супермаркетов



Простой



14 мин



10K

Ретроспектива



+26



13



3



YuriPanchul

14 часов назад

Из обучения школьников цифре на макетке нужно выкинуть все резисторы и вставить микросхемы с FIFO



Простой



3 мин



12K



+25



26



86



inkedsymon
21 час назад

Мамба: архитектура, которая шла убивать трансформеры



Средний



7 мин



11K

Обзор



+23



19



2



runaway_llm
9 часов назад

Слишком просто, чтобы быть правдой? GPT-5.6 выдал доказательство 50-летней математической гипотезы



Простой



4 мин



7.5K

Обзор



+22



8



4



BiktorSergeev
17 часов назад

Worms: как и почему игра про боевых червей уже почти 30 лет остается культовой и популярной



8 мин



10K



+22



3



13



Lunathecat
16 часов назад

Лёгкий кастомный стратокастер из дешёвых комплектующих



Простой



8 мин



9.1K

Тutorial



+21



3



4



IvanKlut
19 часов назад

Новая версия языка разметки текстов Markvan



Средний



4 мин



9.9K

Кейс

**Kova13v**

16 часов назад

Я запретил Claude говорить «всё работает» без пруфов. Это изменило всё



Средний



9 мин



9.2K

Кейс

Из песочницы

Суперсилы и слабости IT-компаний: началось исследование работодателей 2026

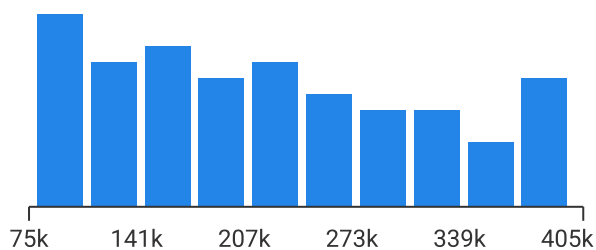
Турбо

[Показать еще](#)

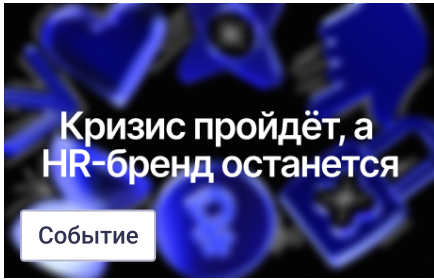
СРЕДНЯЯ ЗАРПЛАТА В IT

216 230 ₽/мес.

— средняя зарплата во всех IT-специализациях по данным из 3 845 анкет, за 2-ое пол. 2026 года. Проверьте «в рынке» ли ваша зарплата или нет!

[Проверить свою зарплату](#)

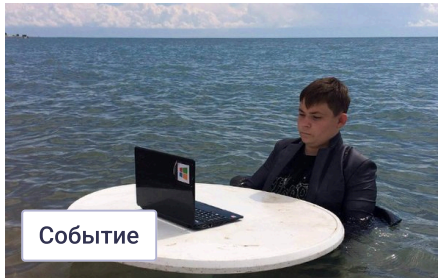
МИНУТОЧКУ ВНИМАНИЯ



Бесплатный вебинар о том, как
строить HR-бренд в 2026 году



Отпуск? Обучение? Курсы? Есть
скидки на все



Лето по расписанию: календарь
событий Хабра

БЛИЖАЙШИЕ СОБЫТИЯ



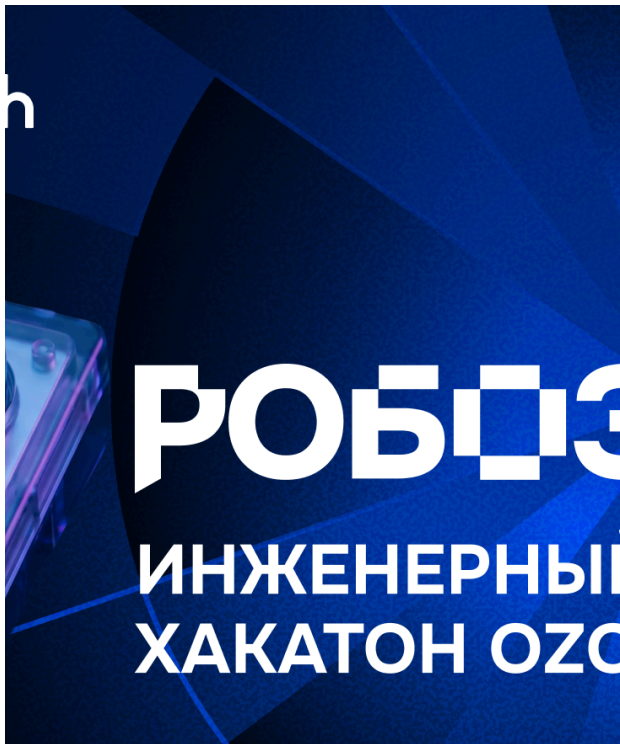
2 марта – 10 августа

**PROBA Awards —
международная
коммуникационная премия,
учрежденная в 2000 году**

Санкт-Петербург

Маркетинг

Больше событий в календаре



11 июня – 2 сентября

**Робозон: хакатон с призовым
фондом 15 млн руб.**

Москва • Онлайн

Разработка

Больше событий в календаре



15 июля

**Веби
эпох**

Онлайн

Марк

Больше



 [Настройка языка](#)

[Техническая поддержка](#)

© 2006–2026, Habr